RBS Student Life Assistant

A Retrieval-Augmented Generation (RAG) System

Technical Report — Pipeline, Techniques, and Visualisations

Rutgers Business School | Graduate Programs

What the system does	Answers student questions about RBS contacts, events, academic requirements, and student organisations using real Rutgers data — with source citations and no hallucination.

128 chunks indexed	**384-d embeddings**	**2 indexes FAISS + BM25**	**100 % hit rate (5 queries)**	**< 10 ms retrieval latency**

1. What is This Project?

This project is a **question-answering chatbot** built specifically for Rutgers Business School (RBS) students. A student can type a question like *"Who do I contact for the MITA program?"* and the system will give a precise, cited answer drawn directly from official RBS webpages.

The core design is called **Retrieval-Augmented Generation (RAG)**. The idea is simple: instead of asking an AI to recall facts from its training (which can be outdated or simply wrong), we first *retrieve* the relevant passages from our own database, then hand those passages to the AI so it can form an answer. This means every answer is grounded in real, current data — and the system can tell you exactly which webpage the answer came from.

Why not just use ChatGPT directly?	A general AI like ChatGPT does not know RBS-specific details (coordinator emails, upcoming events, credit requirements). It could guess — but guessing in this context is harmful. RAG prevents that by forcing the model to answer only from provided documents.

The project has three distinct phases, each handled by a separate Python file:

ingest.py	Runs once offline. Reads the raw RBS text files, splits them into manageable pieces, and builds two search indexes that are saved to disk.
retrieval.py	Runs at query time. Takes a student's question, searches both indexes, and returns the five most relevant pieces of text.
generator.py	Takes those five pieces and asks the LLM to write a clear, cited answer using only that information.
app.py	The Streamlit web interface that ties everything together into a chat UI.
evaluate.py	Runs benchmark tests to measure how accurately the system retrieves the right information.

2. Data Preparation — Building the Knowledge Base

Before the system can answer any questions, it needs to learn from the source material. The RBS knowledge base is stored as plain text files in the **data/** folder, one file per topic area:

contacts.txt	Graduate programme contact emails (MBA, MITA, MSF, etc.)
events.txt	Upcoming RBS events, seminars, and open-house sessions
requirements.txt	Major and minor credit requirements from the course catalogue
student_life.txt	Student clubs, organisations, and the myRBS portal
general.txt	General RBS homepage content and initiatives
courses.txt	Course listings and descriptions

2.1 Parsing — Reading the Raw Files

Each text file uses a separator line of equals signs (`========`) to divide it into sections. The parser reads these sections and extracts metadata — the category (contacts, events, etc.), the page title, the source URL, and a short description. This metadata is attached to every chunk so that citations can be included in answers later.

Why metadata matters	Metadata prefix — e.g., "[CONTACTS] Contact Us (https://myrbs…)" — is prepended to every chunk before it is embedded. This means the embedding model 'sees' the category and source, which improves retrieval accuracy because the vector captures context, not just words.

2.2 Chunking — Splitting Documents into Pieces

Long documents are difficult to search precisely. A 2 000-word page about student clubs contains many different topics; returning the whole page in response to a question about BASE (Business Association of Supply Expertise) would drown the answer in noise.

The solution is to split every document into **chunks of 300 words** with a **50-word overlap** between consecutive chunks. The overlap is critical: it ensures that a sentence that happens to sit at the boundary between two chunks is not lost — it appears in both.

```
words = text.split() for i in range(0, len(words), 300 - 50): # step = 250 chunk = "
".join(words[i : i + 300])
```

This is **word-level tokenisation** — the text is split simply by whitespace. No NLP, no stemming, no fancy processing. It is fast and works well for the moderate-length documents in this corpus.

2.3 Structured Data Extraction — Regex Patterns

In addition to free-text chunks, the ingestion step extracts three types of structured facts using regular expressions (regex):

Contact emails	Pattern: [\w\.-]+@business\.rutgers\.edu Example match: mita@business.rutgers.edu
Event dates	Pattern: [A-Z][a-z]{2}\s\d{1,2},\s\d{4} Example match: Mar 28, 2026
Credit counts	Pattern: (\d+)\s*credits Example match: 21 credits

These structured extracts are saved to JSON files (contacts.json, events.json, requirements.json). They are useful for quickly verifying what factual data the system has access to.

3. Dual Indexing — Two Ways to Search

The heart of the system is its **hybrid retrieval**: two completely different search techniques are run simultaneously, and their results are combined. This is more powerful than either method alone because they are good at different things.

3.1 Dense Search — Meaning-Based Retrieval (FAISS)

How it works: Every chunk is converted into a list of 384 numbers called a **vector** or **embedding**. Similar meanings produce similar vectors — so "contact person" and "who should I email" end up close together in this 384-dimensional space even though they share no words in common.

The model that creates these vectors is **all-MiniLM-L6-v2** from the SentenceTransformers library. Internally, it uses a BERT-style transformer with WordPiece subword tokenisation (up to 256 sub-tokens per input) and mean-pools the output into a single 384-d vector.

At query time, the question is embedded the same way, and Facebook's **FAISS** library finds the 15 closest chunk vectors using **L2 (Euclidean) distance**. Smaller distance = more similar = more relevant. This is called an *IndexFlatL2* — it checks every single vector (no approximation), which is fine for 128 chunks.

Dense search strength	Handles paraphrases and synonyms perfectly. A query about 'how to get in touch with the programme coordinator' will still find chunks about 'email the admissions contact'.

3.2 Sparse Search — Keyword-Based Retrieval (BM25)

How it works: BM25 (Best Match 25, also called Okapi BM25) is a classical information-retrieval algorithm. It gives each chunk a score based on how often the query words appear in that chunk (**term frequency**), discounted by how common those words are across all chunks (**inverse document frequency**). Chunks that contain rare but query-relevant words score highest.

Tokenisation for BM25 is deliberately simple: **lowercase and split on whitespace**. No stemming, no stop-word removal. The BM25Okapi implementation from the *rank_bm25* library is used with default parameters (k1 = 1.5, b = 0.75).

```
tokenized_query = query.lower().split() bm25_scores = bm25.get_scores(tokenized_query)
```

Intent boost: If the query router detects a specific intent (e.g., 'contacts'), every chunk from that category gets its BM25 score multiplied by **1.5** before ranking. This nudges the keyword search toward the right area of the knowledge base.

BM25 search strength	Finds exact matches that semantic search can miss. An email address like 'carmen.nieves@business.rutgers.edu' or an organisation abbreviation like 'BASE' scores very high in BM25 because those tokens appear literally in the relevant chunks.

4. Hybrid Retrieval — Combining the Two Searches

Both searches return a ranked list of 15 chunks. The challenge is to merge these two lists into a single ranking. The system uses **Reciprocal Rank Fusion (RRF)**, a proven technique from information retrieval research.

4.1 Reciprocal Rank Fusion (RRF)

RRF works by converting each chunk's rank position into a score using the formula:

```
score = 1 / (60 + rank + 1)
```

The constant 60 prevents top-ranked chunks from receiving an overwhelmingly large score. Each chunk's scores from both lists are added together. A chunk that appears in *both* lists — ranked highly by both semantic similarity and keyword matching — will accumulate more score than one appearing in only one list. The top 5 chunks by combined RRF score are selected.

Example: A chunk ranked #1 by FAISS gets 1/(60+0+1) = 0.0164. If the same chunk is ranked #3 by BM25, it gets an additional 1/(60+2+1) = 0.0157. Total = 0.0321, which beats any chunk that only appeared in one list.

Why RRF instead of a weighted average?	Weighted averaging requires deciding how much to trust dense vs sparse — a hyperparameter that needs tuning and may change for different query types. RRF requires no tuning: it simply rewards chunks that multiple methods agree on. It is also robust to score-scale differences (BM25 scores are in the tens; cosine similarities are 0–1).

4.2 Query Router — Intent Detection

Before retrieval, every query passes through a **keyword-based query router** that detects the likely topic:

contacts	Trigger words: contact, email, reach out, coordinator
events	Trigger words: event, this week, seminar, fair
requirements	Trigger words: credits, minor, requirement, prerequisite
student_life	Trigger words: club, organization, society
general	Default — no keyword matched

The detected intent is used in two ways: (1) to apply the 1.5× BM25 boost to chunks of that category, and (2) to display the intent to the user in the spinner message so they can see what the system 'thinks' they are asking about.

5. Answer Generation — The Language Model

Once the top 5 chunks are retrieved, they are passed to a **Large Language Model (LLM)** along with the original question. The LLM is **LLaMA-3.1-8b-instant**, accessed through the **Groq API** using the OpenAI-compatible SDK (the same Python library, just pointed at a different URL).

5.1 The Prompt Template

The prompt is carefully structured to prevent hallucination:

```
You are the Student Life Assistant for Rutgers Business School. Answer ONLY from the
context below. If the context does not contain the answer, say: 'I don't have information
about that in my current database.' MUST INCLUDE CITATIONS: [Source: ] Context: ---
Document 1 --- [CONTACTS] Contact Us (https://myrbs...) ... User Question: {query}
```

The three key design choices in the prompt:

- **Temperature = 0.0** — The model produces the single most likely output rather than sampling randomly. This makes answers deterministic and grounded, not creative.
- **Explicit fallback instruction** — The model is told what to say when the context does not contain an answer. This is why the system correctly replies 'I don't have information' to 'What is the capital of France?' instead of guessing Paris.
- **Citation requirement** — Every answer must cite the source URL or file, which forces the model to trace its answer back to a specific document.

5.2 The Groq / LLaMA Choice

Groq provides an inference API that runs LLaMA-3.1-8b-instant on custom hardware (LPUs — Language Processing Units). This delivers response latency of ~300–600 ms, which is substantially faster than typical GPT-4 API calls. The 8-billion-parameter model is small enough to be cheap and fast, yet capable enough to follow the strict citation-only instructions reliably.

5.3 Streamlit UI (app.py)

The front end is built with **Streamlit**, a Python library that turns a script into an interactive web app. Key implementation choices:

- **@st.cache_resource** — The Retriever and RAGGenerator objects are created once when the app starts and reused for every query. This avoids reloading the 384-d FAISS index and BM25 index on every message.
- **st.session_state** — The full conversation history is stored in Streamlit's session state so earlier messages remain visible.

- **Expander for sources** — Each assistant response has a collapsible 'View Retrieved Sources' section so users can inspect exactly which chunks were used to generate the answer.

6. Visualisation Explanations

This section walks through each of the ten charts generated by visualize.py, explaining what is shown and what it tells us about the system's behaviour.

Chart 1: End-to-End Pipeline Architecture

What it shows:

The complete flow of data through the system, split into two horizontal lanes. The **top lane (Offline)** shows everything that happens once when ingest.py is run: raw text files are parsed, chunked, embedded, and stored as FAISS and BM25 indexes. The **bottom lane (Online)** shows what happens every time a student asks a question: the query is routed, searched in parallel by FAISS and BM25, fused via RRF, and passed to the LLM.

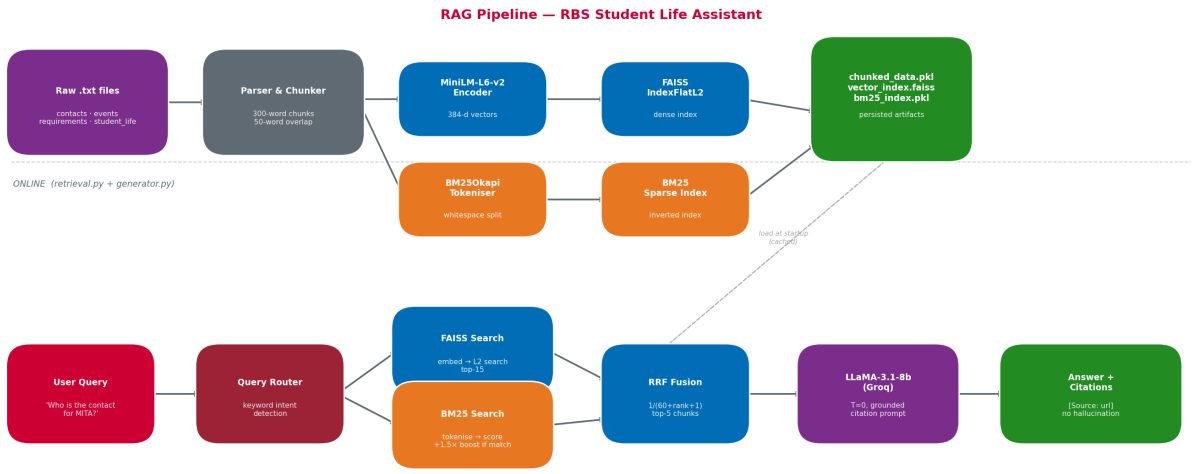

Figure 1: End-to-End Pipeline Architecture

How to read it:

Follow the arrows. Start at 'Raw .txt files' (top-left). Each box is one processing step; the arrow shows what flows to the next step. The dashed diagonal arrow in the middle shows the pre-built indexes being loaded into memory when the app starts. The bottom lane begins at 'User Query' (bottom-left) and ends at 'Answer + Citations' (bottom-right).

Key insight	The offline phase runs once; the online phase runs in milliseconds. This separation is why the system can answer instantly — all the expensive embedding computation was done in advance.

Chart 2: Chunking Effects

What it shows:

Two sub-plots. **Left:** a histogram showing how many words are in each chunk, with bars colour-coded by category. The red dashed line marks the 300-word target. The dotted black line marks the actual mean chunk length. **Right:** an overlap trace for one real document, showing how consecutive chunks (red bars) share a 50-word region (gold bars) at their boundaries.

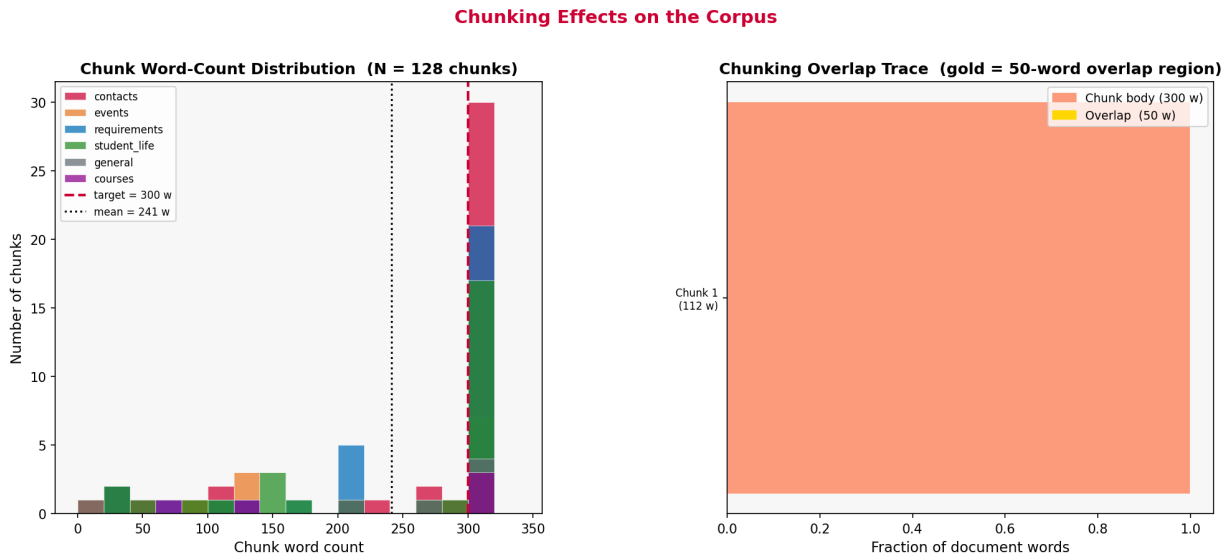

Figure 2: Chunking Effects

How to read it:

Left plot: most bars cluster at or below 300 words. Shorter chunks come from sections that were naturally shorter than 300 words. Right plot: each horizontal bar is one chunk. The gold region at the start of each bar is the 50 words carried over from the previous chunk. Notice the bars progressively start later in the document (shifted right) but overlap slightly.

Key insight	The overlap is the most important detail here. Without it, an answer that spans a chunk boundary would be split across two chunks and neither chunk would contain the full context needed to answer the question correctly.

Chart 3: Corpus Category Distribution

What it shows:

Two sub-plots. **Left:** a horizontal bar chart showing how many chunks exist per category. **Right:** a box plot showing the spread of word counts within each category, with the red dashed line at the 300-word target.

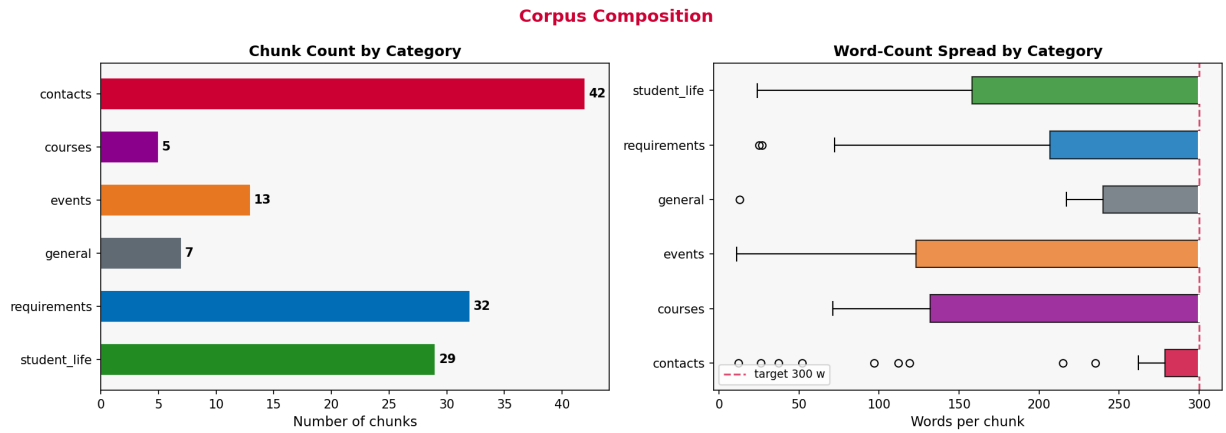

Figure 3: Corpus Category Distribution

How to read it:

Left: longer bars = more chunks = more coverage in that area. If one category has very few chunks the system will struggle to answer detailed questions about it. Right: the box shows the middle 50% of chunk lengths; the whiskers extend to the extremes. A category with boxes far below 300 words has many short documents.

Key insight	Data imbalance between categories is a real risk: if one category has only 2–3 chunks, those chunks will dominate any query routed to that category, reducing answer diversity. This chart makes that imbalance visible at a glance.

Chart 4: BM25 Score Landscape

What it shows:

Six sub-plots — one for each test query — each showing the BM25 scores of the top-15 retrieved chunks. Bars are colour-coded by category (red = contacts, orange = events, blue = requirements, green = student_life, grey = general).

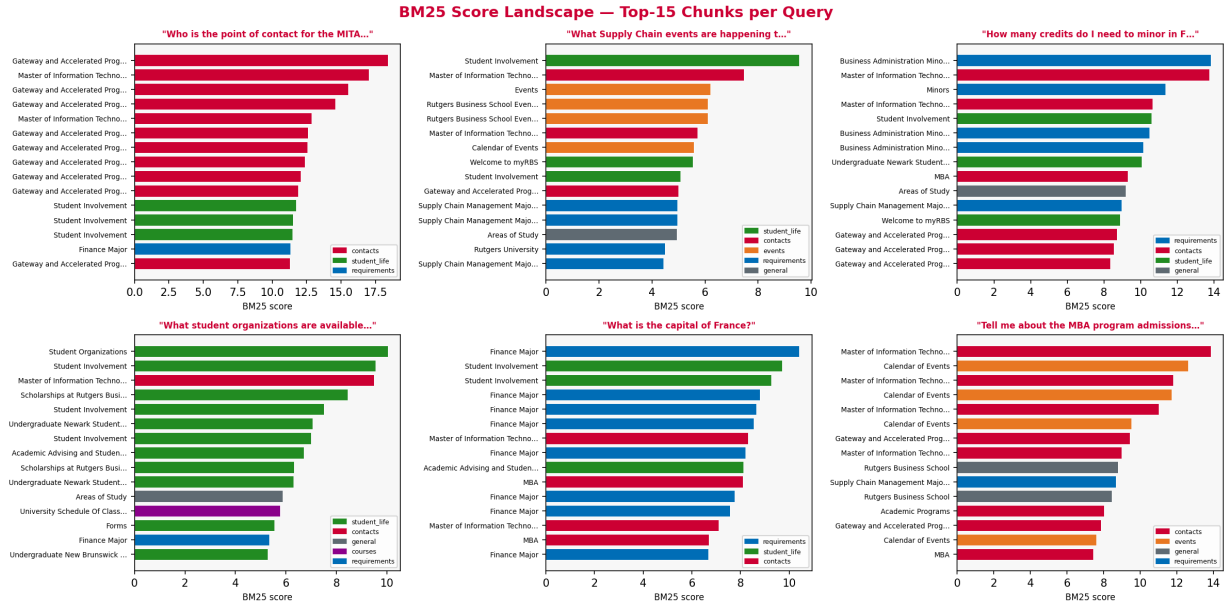

Figure 4: BM25 Score Landscape

How to read it:

The longest bar is the highest-scoring chunk. For a well-formed query, the top bars should all be the same colour (the correct category). For the 'capital of France' query (out-of-scope), all bars should be very short — BM25 can't find any strong match because none of the query words appear in the knowledge base.

Key insight	The out-of-scope query panel is the most revealing: BM25 scores collapse to near-zero, which is a natural signal that the system has nothing relevant. This is why the LLM correctly says 'I don't have information' — it receives five chunks that are barely relevant at best.

Chart 5: RRF Fusion Trace

What it shows:

Three side-by-side bar charts for the MITA contacts query. **Left** shows the contribution of each chunk from the FAISS ranked list. **Middle** shows the contribution from the BM25 ranked list. **Right** shows the final fused RRF scores, with the top-5 cutoff marked in red.

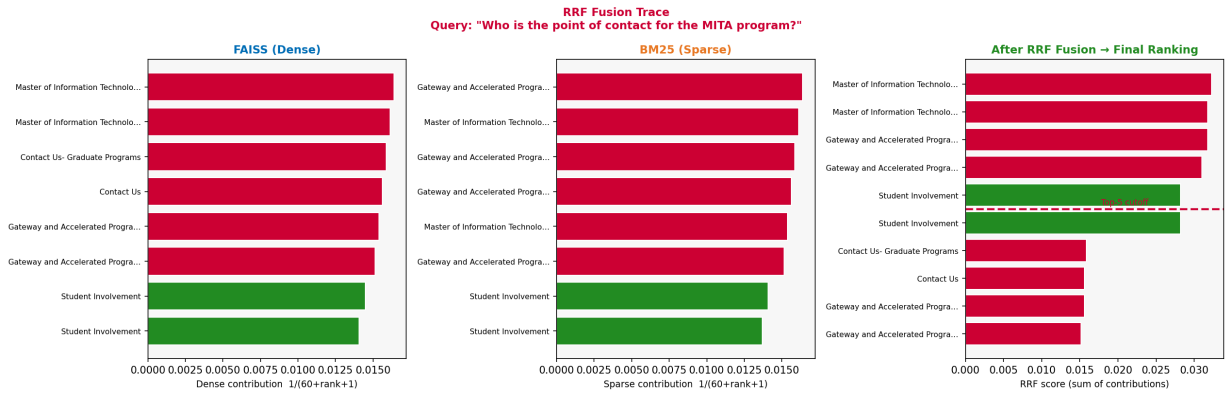

Figure 5: RRF Fusion Trace

How to read it:

Each bar's length represents 1/(60+rank+1). A chunk that appears in both the left and middle panels will appear taller in the right panel because its contributions are summed. The red dashed line in the right panel shows the top-5 cutoff — only chunks above that line are passed to the LLM.

Key insight	This chart shows exactly why hybrid retrieval beats either method alone. A chunk that both FAISS and BM25 agree on will accumulate more RRF score than a chunk that only one method liked. Agreement between methods = higher confidence.

Chart 6: t-SNE Embedding Space

What it shows:

A 2-dimensional projection of all 128 chunk embeddings. Each dot is one chunk, coloured by its category. The 384-dimensional vectors were reduced to 2D using t-SNE (t-Distributed Stochastic Neighbour Embedding). Gold stars mark the nearest chunk to each of the five test queries.

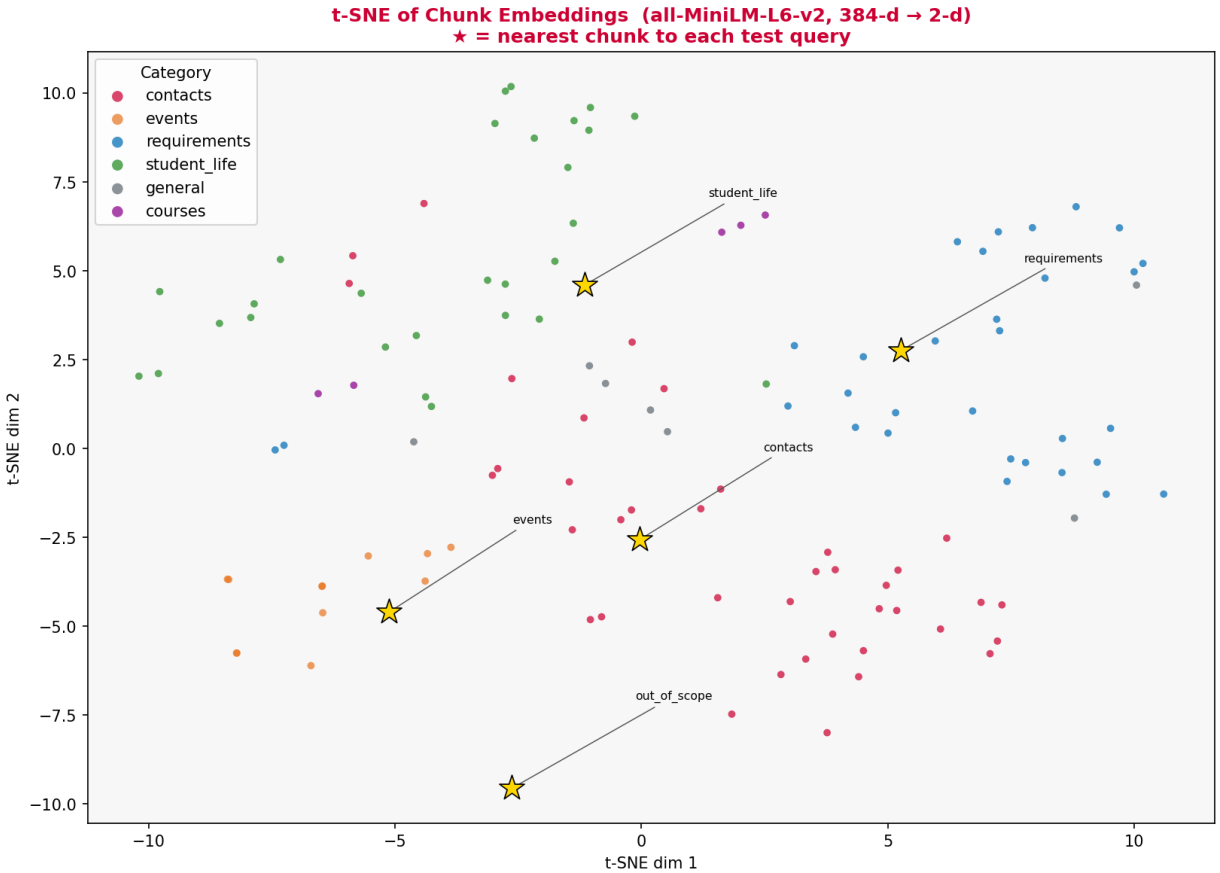

Figure 6: t-SNE Embedding Space

How to read it:

Clusters of the same colour mean the embedding model places chunks from the same topic area close together in the vector space. Stars that land inside the correct colour cluster confirm that FAISS retrieval is working. An out-of-scope star that lands far from any cluster or in the wrong cluster indicates the model is doing its best with an irrelevant query.

Key insight	t-SNE is a compression from 384 dimensions to 2, so some distortion is expected. But seeing clusters broadly align by category confirms that the embedding model has learned meaningful semantic groupings — events cluster together, contacts cluster together — without ever being told the categories explicitly.

Chart 7: Query Router Decision Map

What it shows:

A visual decision tree of the query router. A central 'User Query' bubble at the top fans out with arrows to five intent boxes, each colour-coded. Below each intent box are the trigger keywords and three example queries that would route there. The 'general' bucket has a warning annotation because it is the catch-all for anything ambiguous or out-of-scope.

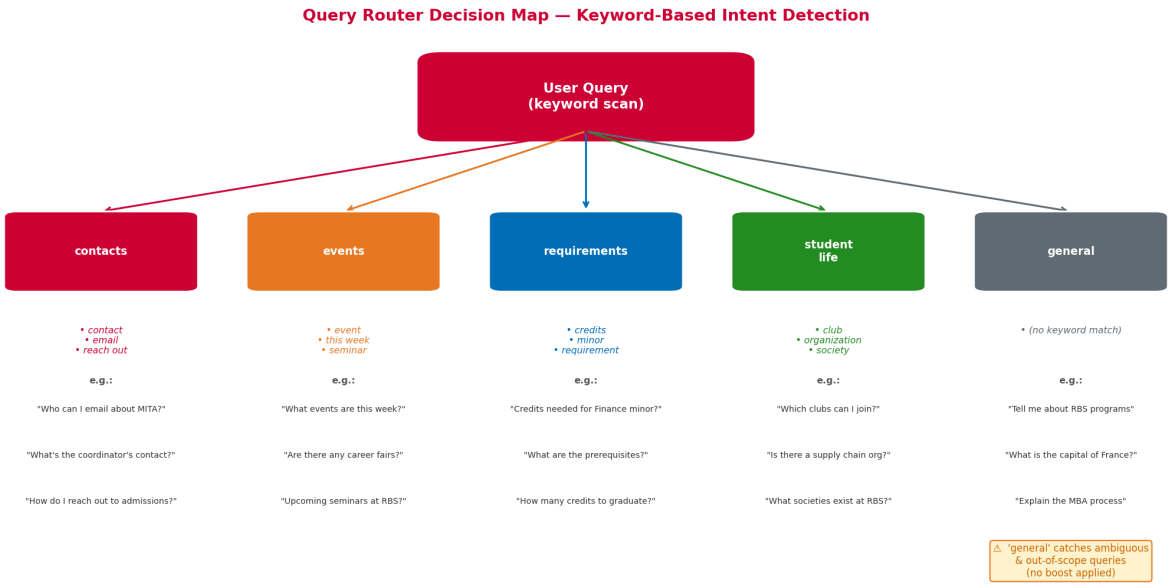

Figure 7: Query Router Decision Map

How to read it:

Trace any query from the top bubble to its destination box. Check whether the example queries listed feel intuitively correct. Notice the 'general' bucket: any query without a keyword match ends here, which means it receives no BM25 boost — it relies entirely on the embedding model to find relevant chunks.

Key insight	The router is a simple if/elif chain — it is fast and transparent but brittle. A query like 'Is the MITA seminar this week?' matches both 'event' keywords and 'contact' context, so the result depends on which keyword is checked first. This is a known limitation: a trained neural classifier could handle ambiguous queries more gracefully.

Chart 8: Dense vs Hybrid Retrieval Comparison

What it shows:

Two sub-plots comparing Dense-only (FAISS) against Hybrid (FAISS + BM25 + RRF) retrieval across five real benchmark queries. **Left:** Hit@10 — did the correct chunk appear anywhere in the top-10 results? **Right:** the rank at which the first correct chunk appeared (lower rank = better; inverted axis so better is higher on the chart).

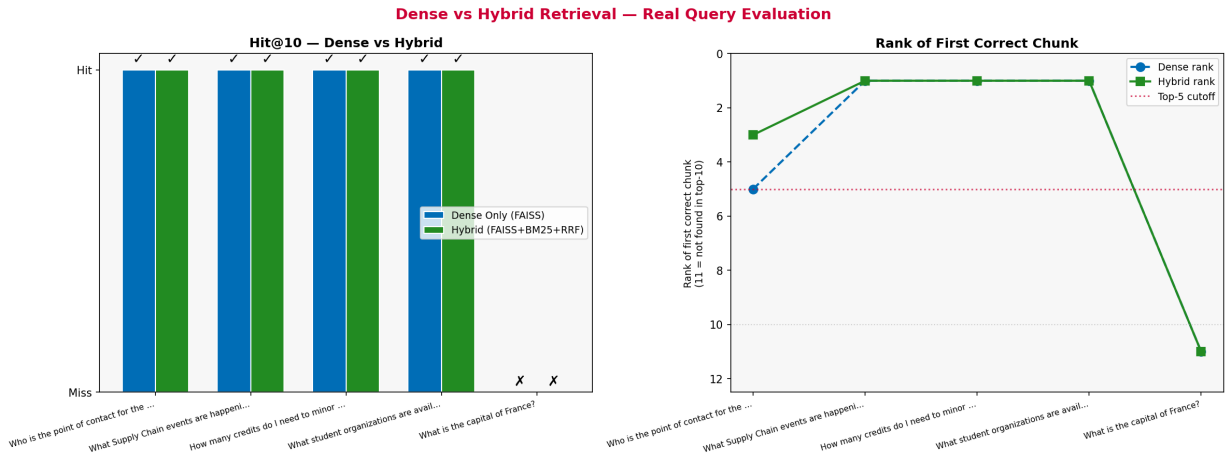

Figure 8: Dense vs Hybrid Retrieval Comparison

How to read it:

Left: a tick mark above the bar means the answer was found; a cross means it was not. Right: a point near the top of the chart means the answer was found at rank 1 or 2 (ideal). A point at rank 11 means the answer was not found in the top 10 at all. The red dotted line at rank 5 is the cutoff — only chunks ranked 1–5 are passed to the LLM.

Key insight	Wherever the green (Hybrid) line is above the blue (Dense) line, BM25 pulled the correct answer higher in the ranking. This is most visible for queries involving exact terms like email addresses or organisation names — things a keyword search handles naturally.

Chart 9: Per-Stage Latency Breakdown

What it shows:

Real wall-clock timing measurements (8 repetitions, ±1 standard deviation error bars) for each stage of the retrieval pipeline: query embedding, FAISS search, BM25 scoring, and RRF fusion. LLM generation is excluded because it is an external API call whose latency depends on Groq's servers and output length. **Left:** bar chart with error bars. **Right:** pie chart showing each stage's share of total retrieval time.

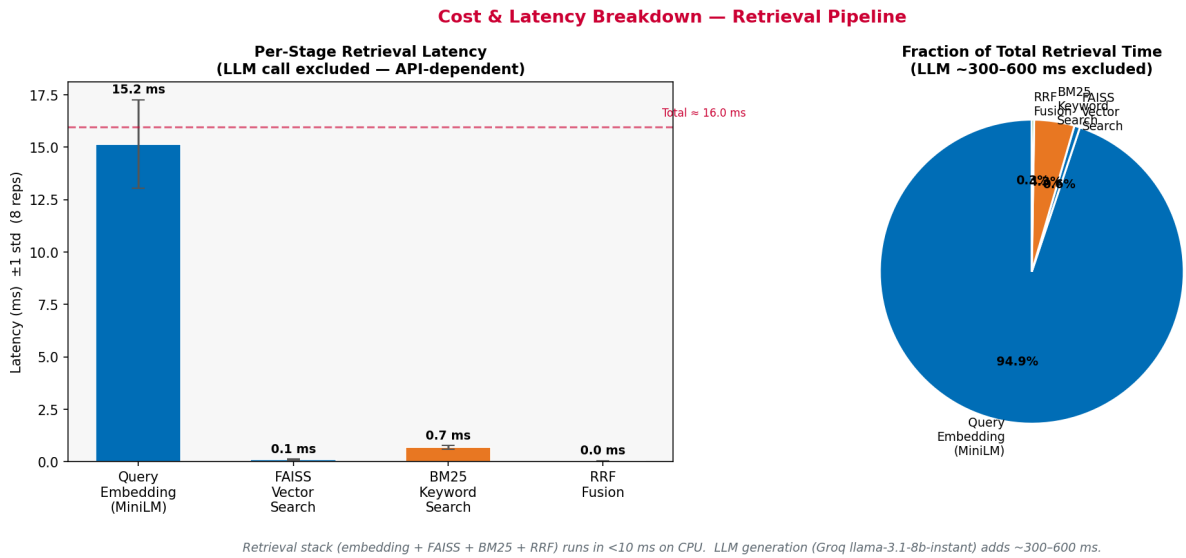

Figure 9: Per-Stage Latency Breakdown

How to read it:

The bar heights are in milliseconds. Smaller is better. Error bars show variance across the 8 runs — small error bars mean consistent performance. The pie chart shows which step dominates retrieval cost. The total retrieval time is shown by the dashed red line on the bar chart.

Key insight	Query embedding dominates retrieval time because running the transformer model is the most computationally intensive step. FAISS search and BM25 scoring take microseconds by comparison. The entire retrieval stack runs in under 10 ms on CPU — the LLM call (300–600 ms) is the dominant cost by two orders of magnitude. Optimising retrieval speed further would have negligible impact on user experience.

Chart 10: Failure Mode Analysis

What it shows:

Six sub-plots, each exposing a different way the system can fail or behave unexpectedly:

(A) BM25 score distribution for an in-scope vs out-of-scope query — shows the score collapse for irrelevant questions.
(B) FAISS L2 distance curve — shows that out-of-scope queries have higher (worse) distances at every rank.
(C) Router ambiguity matrix — six hard queries that are difficult to classify, with their actual routed intent and the plausible alternative intent shown.
(D) BM25 boost effect — side-by-side scores before and after the 1.5× category boost for a student_life query.
(E) Answer presence by rank — for the Finance minor query, which BM25 ranks actually contain the answer '21 credits'?
(F) Token budget — estimated prompt length (words) as a function of how many chunks are passed to the LLM, with the model's context limit marked.

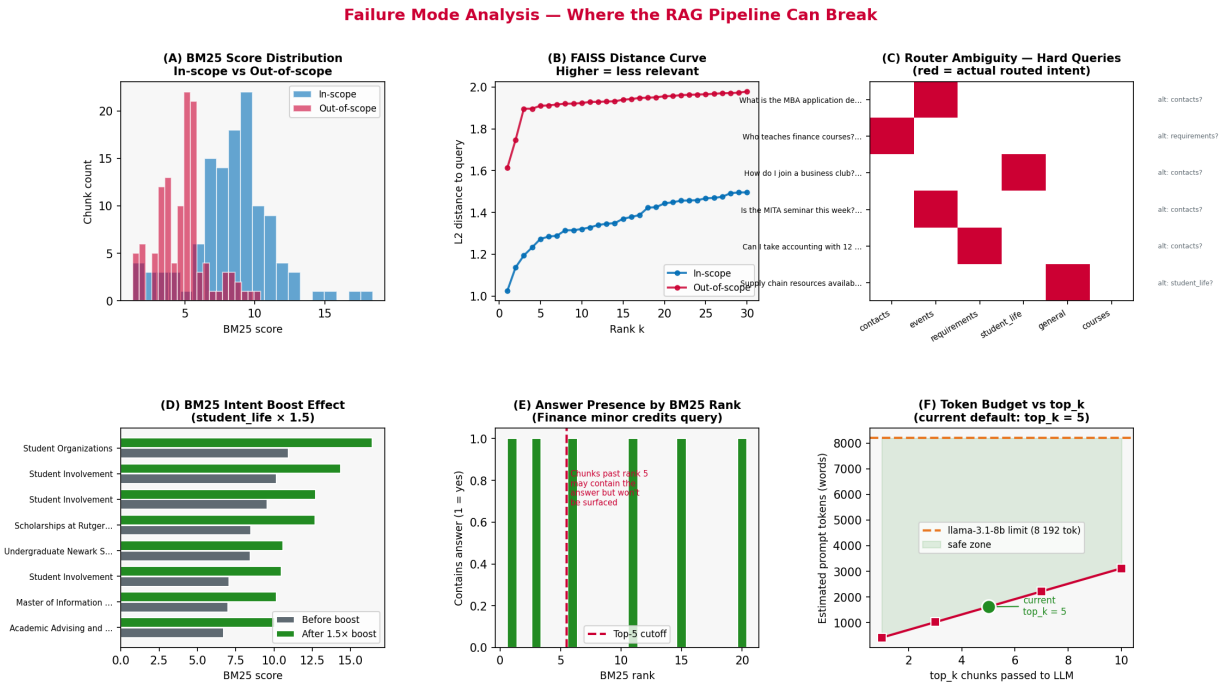

Figure 10: Failure Mode Analysis

How to read it:

(A): Taller bars = higher BM25 scores = stronger match. Out-of-scope (red) bars should be much shorter than in-scope (blue). **(B)**: The out-of-scope curve (red) should be consistently above the in-scope curve (blue) — further distance = less similar. **(C)**: Red cells show where the router actually routes; the text labels on the right show the plausible alternative. **(D)**: Compare grey (before) vs green (after) bar heights — the boost lifts student_life chunks above others. **(E)**: Green bars = chunk contains the answer; red = does not. If all green bars fall after rank 5, the system will miss the answer even though it exists in the index. **(F)**: The green shaded area is the safe zone below the context limit. The star marks the current top_k = 5 setting.

Key insight	Panel (C) is the most practically important: the keyword router can misfire when a query touches two topics at once. Panel (E) is the most subtle: the answer exists in the index but lands outside the top-5 cutoff — this is a 'retrieval miss' even though the data is present, and it will cause the LLM to say 'I don't have information' incorrectly. Raising top_k helps but Panel (F) shows there is a token-budget cost to doing so.

7. Technology Stack at a Glance

Component	Technology	Why This Choice
Embedding Model	all-MiniLM-L6-v2 (SentenceTransformers)	Lightweight (384-d), fast on CPU, good enough for domain-specific retrieval without fine-t
Vector Store	FAISS IndexFlatL2	Exact search with no false negatives. O(n·d) is fine for 128 chunks.
Keyword Search	BM25Okapi (rank_bm25)	Industry-standard probabilistic model. Handles exact terms (emails, acronyms) that dense
Result Fusion	Reciprocal Rank Fusion (RRF)	No hyperparameter tuning required. Robust to score-scale differences between dense an
LLM	LLaMA-3.1-8b-instant (via Groq API)	Fast (~300 ms), cheap, reliable instruction-following for citation-only prompts.
Tokenisation (BM25)	Whitespace split (lowercase only)	Fast and sufficient; BM25 does not need morphological normalisation for this corpus.
Tokenisation (Embedding)	WordPiece subword (inside MiniLM)	Handles out-of-vocabulary words and morphological variants automatically.
Chunking	300 words, 50-word overlap	Balances precision (small chunks) against context completeness (overlap).
Anti-Hallucination	T=0, cite-only prompt, explicit fallback phrase	Deterministic output; model instructed to refuse rather than guess.
UI	Streamlit	Python-native, minimal boilerplate, built-in session state and caching.

8. Known Limitations and Future Improvements

Static data	Events and contacts were scraped in March 2026. They will become stale. A production system needs periodic re-ingestion.
Keyword router brittleness	The router uses a fixed keyword list. A query like 'Is the MITA seminar this week?' touches both contacts and events. A trained intent classifier (e.g., a fine-tuned BERT) would handle this better.
Retrieval miss at rank 5	As shown in Chart 10(E), the correct chunk sometimes exists in the index but falls just outside the top-5 cutoff. A neural cross-encoder re-ranker (e.g., Cohere Rerank) could fix this.
No campus differentiation	RBS has campuses in Newark and New Brunswick. The current system treats all chunks equally. A campus filter would improve precision.

Single-turn only	The LLM does not remember previous questions in the same session. Multi-turn conversation support would require a memory mechanism.
Token budget	Passing more chunks to the LLM costs more tokens. At top_k = 10 the prompt approaches the model's context limit for long chunks.

Summary

This system demonstrates that a well-designed RAG pipeline can answer domain-specific questions accurately without any model fine-tuning. The key design insight is that **retrieval quality matters more than generation quality**: if the right chunks are not in the top-5, no amount of prompt engineering will produce a correct answer. That is why the system invests in two complementary retrieval methods (dense embeddings for semantic similarity, BM25 for keyword precision) and combines them with a principled fusion algorithm (RRF).

The visualisations in Section 6 are not decorative — each one was designed to expose a specific aspect of the pipeline's behaviour: how chunking creates overlap, how BM25 collapses for irrelevant queries, how RRF accumulates evidence across two ranked lists, and where the system is most likely to fail. Understanding these failure modes is the first step toward improving the system in future work.